

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 8 OF 9

TESTING & QA SPECIFICATION

Test Strategy - Security & Load Testing - Multi-Tenancy Integrity - UAT - Release Gates

Field	Value
Document Status	Draft - v0.1 - Phase 8 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 1 Value Streams & Module Scope; Vol. 2 Sequence Diagrams; Vol. 3 Coding Standards & Performance Targets; Vol. 4 RLS; Vol. 5 Resilience Patterns; Vol. 6 Security Test Requirements; Vol. 7 CI/CD Pipeline & Environments
Downstream Volumes	Vol. 9 UI/UX & API Design Document

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume defines the complete test strategy needed to ship the platform with confidence: the test pyramid and ownership model, concrete testing standards for each layer (unit, integration, contract, end-to-end), the formalized security test plan promised by Vol. 6 Section 12, performance/load testing against Vol. 3's targets, the multi-tenancy and financial-integrity test suites that protect the platform's highest-severity risks, resilience/chaos testing for Vol. 5's integration failure modes, test data management, user acceptance criteria tied to Vol. 1's module scope, and the defect/release-gate process that plugs into Vol. 7's CI/CD pipeline.

1.2 Testing Philosophy

Given the modular-monolith-first decision (Vol. 2 Section 2.1) and the financial-integrity stakes (Vol. 1 Principle P4), this platform favors a test pyramid weighted toward fast, deterministic unit and integration tests, with a thin layer of end-to-end tests reserved for the value streams where cross-module correctness cannot be verified any other way (Section 6). Manual exploratory testing is retained deliberately for UI/UX judgment calls (Vol. 9) that automation cannot meaningfully assess.

2. Test Strategy & Pyramid

2.1 Pyramid Shape

Test pyramid (approximate proportion of total test count)

flowchart TB
 E2E["E2E / UAT - ~5% - Section 6, 12"]
 Contract["Contract Tests - ~10% - Section 5"]
 Integration["Integration Tests - ~25% - Section 4"]
 Unit["Unit Tests - ~60% - Section 3"]
 E2E --- Contract --- Integration --- Unit

2.2 Ownership Model

Test Layer	Owned By	Runs
Unit	Feature engineer authoring the change, per bounded context (Vol.2 Section 3.2)	Every commit, local + CI (Vol.7 Section 5.1)
Integration	Feature engineer + module owner	Every pull request, ephemeral CI environment (Vol.7 Section 2.1)
Contract	API/event producer and consumer teams jointly	Every pull request affecting a public contract
End-to-end	QA function (or rotating engineering ownership pre-dedicated-QA-hire)	Every merge to main, against staging (Vol.7 Section 2.1)
Security	Security-conscious engineer + Vol.6-defined gates	Per Vol.6 Section 12 cadence (CI build, weekly, pre-GA)
Performance/load	Platform/infra owner	Pre-release and on-demand against staging

3. Unit Testing Standards

3.1 Coverage Targets

Module Category	Coverage Target	Rationale
Ledger, Receivables, Payables domain logic (CTX-02/03/04)	90%+ line coverage on domain services; 100% on the posting/state-machine logic (Vol.2 Section 8.1 invoice lifecycle, double-entry validation)	Highest financial-correctness risk (Vol.1 Principle P4)
Other core modules (Inventory, Banking, Tax, Projects)	80%+	High business value, lower correctness risk than the ledger itself
Supporting/generic modules (Documents, Collaboration, Automation)	70%+	Lower risk; over-testing here has diminishing returns
Cross-cutting infrastructure (caching, logging utilities, Vol.3 Section 6-7)	80%+, with explicit tests for the tenant-key-prefix enforcement (Vol.3 Section 7.2)	A bug here risks the cross-tenant leakage failure mode (Vol.6 Section 2.2)

3.2 What Must Be Unit Tested

- Every CS-03 Money-type arithmetic path (Vol.3 Section 4.2) - explicit tests for rounding, currency mismatches, and negative-amount rejection.
- Every state transition in a domain state machine (e.g. Vol.2 Section 8.1 invoice lifecycle) - both valid and invalid transition attempts.
- Every validation rule that produces a namespaced error code (Vol.3 Section 4.3) - one test per error code, asserting the exact code is returned.
- Every branch of a business rule with a Vol.1 BC-xx-numbered acceptance criterion (Section 12 traces these).

4. Integration Testing

4.1 Scope

Integration tests verify a module's interaction with its own database schema (Vol.4), its published/consumed domain events (Vol.2 Section 10), and its calls into other modules' public APIs (Vol.2 Section 11) - run against the ephemeral CI environment (Vol.7 Section 2.1) with real PostgreSQL, Kafka, and Redis instances (via containers), not mocks, since the highest-value bugs in this platform live at exactly these boundaries.

4.2 Required Integration Test Scenarios (representative)

Scenario	Verifies	Traces To
Invoice sent triggers correct journal posting	Receivables -> Ledger event flow produces the expected debit/credit pair	Vol.2 Section 9.1 sequence diagram
Payment received matches and updates invoice balance	Receivables + Banking reconciliation flow	Vol.2 Section 9.2
3-way match approves or flags a bill correctly	Payables + Inventory event choreography	Vol.2 Section 9.3
RLS blocks cross-tenant read/write at the repository layer	Data-layer tenant isolation (layer 1 of Vol.6 Section 8 defense-in-depth)	Vol.4 Section 7
Idempotency key prevents duplicate financial mutation on retry	Vol.3 Section 5 API convention + Vol.5 Principle I4	Vol.3 Section 5, Vol.5 Section 2
Outbox relay publishes an event exactly once per committed transaction, even after a simulated relay crash	Vol.5 Section 8.1 outbox pattern reliability	Vol.5 Section 8.1

5. Contract Testing

5.1 API Contract Tests

Every public API endpoint (Vol.3 Section 5, Vol.2 Section 11) has a contract test asserting its request/response schema matches its published OpenAPI definition (Vol.3 Section 4.2 CS-05) - run in CI on every change to either the endpoint implementation or the schema, catching drift before it reaches an external integrator (Vol.5 Section 7).

5.2 Event Schema Contract Tests

Every domain event (Vol.2 Section 10 catalog) has a schema-registry-backed contract test (Vol.3 Section 4.2 CS-06): a producer cannot publish a payload that violates the registered schema, and a new schema version must be backward-compatible (additive only) unless explicitly declared as a new major version. This is what makes the event-driven architecture in Vol. 2 Section 2.2 safe to evolve without breaking every downstream consumer simultaneously.

5.3 Consumer-Driven Contract Verification

For the highest-fan-out events (JournalPosted, InvoiceSent - Vol.2 Section 10), each consumer module publishes its own expectations of the event shape it depends on; the producer's CI pipeline runs all registered consumer contracts before merge, catching a breaking producer change even if the consumer's own repository/module hasn't been touched.

6. End-to-End Testing

E2E tests are scoped to the six value streams defined in Vol. 1 Section 7 - each value stream gets one or more E2E scenarios exercised through the real API surface against staging (Vol.7 Section 2.1), not through internal module calls.

Value Stream	E2E Scenario	Persona Simulated
VS-01 Order-to-Cash	Create quote -> convert to invoice -> send -> customer pays via portal -> reconcile -> verify ledger balance	PER-04, PER-09
VS-02 Procure-to-Pay	Raise PO -> receive goods -> vendor submits bill via portal -> 3-way match -> pay -> reconcile	PER-05, PER-10
VS-03 Record-to-Report	Post transactions across a period -> lock period -> generate financial statements -> verify report figures against known input	PER-02
VS-04 Project-to-Bill	Log timesheets -> approve -> bill client -> verify invoice line items match logged hours	PER-07
VS-06 Close-the-Books	Full month-end close simulation -> accountant review -> approve -> verify period lock prevents further posting	PER-02, PER-03

VS-05 (Hire-to-Pay/Payroll) is excluded from the Release 1 E2E suite per Vol. 1 Section 6's R3 scoping and will be added when Payroll (CTX-09) is built.

7. Security Testing (Formalized from Volume 6)

Volume 6 Section 12 specified which security tests must exist; this section is their concrete test-plan detail and CI wiring.

Test	Detail	Gate
Cross-tenant leakage suite	For every RLS-protected table (Vol.4 Section 7): authenticate as tenant A, attempt every read/write operation against tenant B's known record IDs, assert zero rows/access in every case	Blocks merge on any failure - zero tolerance
SAST	Static analysis across the monorepo (Vol.3 Section 3.1) for OWASP Top 10 patterns and the Vol.3 Section 4.2 CS rules (e.g. CS-03 float-for-money misuse)	Blocks merge on high/critical findings
Dependency scanning	npm audit-equivalent against every package in the monorepo	Blocks merge on known-critical CVEs; alerts (non-blocking) on medium
DAST	Automated scan of the public API surface and web app against staging	Weekly; findings triaged within 5 business days
Secrets-hygiene scan	Pre-commit + CI scan for accidental credential commits (Vol.6 Section 6.3)	Blocks commit/merge on any match
Penetration test	Third-party engagement covering integration surfaces (Vol.5) and multi-tenant isolation (Vol.6 Section 8)	Required before Release 1 GA sign-off; annually thereafter
Webhook/signature bypass testing	Attempt to submit unsigned or badly-signed webhooks to every inbound receiver (Vol.5 Section 3.4, Section 7.1)	Required before Release 1 GA

8. Performance & Load Testing

Verifies the Vol. 3 Section 12 performance targets under the Vol. 7 Section 7 scaling design, run against a staging environment sized to a known fraction of production capacity so results can be extrapolated.

Test	Target Verified	Method
Core API load test	P95 < 300ms for CRUD operations (Vol.3 Section 12)	Synthetic load generator ramping to the Release 1 concurrent-tenant target (1,000+), sustained for 30 min
Reporting query load test	P95 < 2s for custom report generation (Vol.3 Section 12)	Concurrent report-generation requests against a seeded large-dataset tenant
Event processing lag test	< 5s P95 event-to-consumer-completion (Vol.3 Section 12)	Inject a burst of domain events and measure consumer lag under the Vol.7 Section 7 Kafka sizing
Recurring invoice batch test	10,000-tenant nightly batch completes within a 1-hour window (Vol.3 Section 12)	Simulated batch run against a seeded multi-tenant dataset in staging
Soak test	No memory/connection leak over sustained operation	48-hour sustained moderate load prior to GA
Spike test	Autoscaling (Vol.7 Section 7) responds correctly to sudden load increase	Abrupt 10x traffic increase, verify scale-out and no dropped requests beyond acceptable error budget

9. Multi-Tenancy & Financial Data Integrity Testing

9.1 Multi-Tenancy Suite

Beyond the cross-tenant leakage suite (Section 7), this includes cache-key isolation tests (Vol.3 Section 7.2 tenant-prefixed keys), tenant-context-propagation tests (Vol.3 Section 11.2 - verifying every async job and event consumer correctly carries tenantId, not just synchronous API requests), and tenant-deletion/offboarding tests (Vol.4 Section 10) verifying a deactivated tenant's data is inaccessible without being prematurely purged.

9.2 Financial Integrity Suite

- Double-entry balance invariant: after every test scenario (unit, integration, or E2E) that posts to the ledger, assert total debits equal total credits for every affected journal entry and, in aggregate, for the tenant's full ledger.
- Append-only enforcement: attempt direct UPDATE/DELETE against ledger.journal_lines using the application's database role and assert it is rejected (Vol.4 Section 4.1 principle D2).
- Reversal correctness: for every voided/reversed transaction, assert the reversing entry exactly negates the original and that both remain queryable (never physically deleted).
- Idempotent replay: replay the same domain event twice through a consumer and assert no double-posting occurs (Vol.3 Section 8.2 idempotent-consumer requirement).
- Audit hash-chain integrity: run the periodic verification job (Vol.6 Section 9.2) against a seeded audit log and assert it detects an artificially tampered entry.

10. Resilience & Chaos Testing

Verifies the failure-handling patterns from Vol. 5 Section 8 and the disaster-recovery posture from Vol. 7 Section 9 actually work, rather than trusting the design on paper.

Test	Failure Injected	Expected Behavior
Circuit breaker trip	Simulated third-party provider (bank aggregator/payment gateway, Vol.5) returns errors above threshold	Circuit opens (Vol.5 Section 8.2), calls fail fast, platform remains responsive for unrelated operations
Kafka broker unavailability (brief)	MSK broker restart/failover (Vol.7 Section 7)	Outbox pattern (Vol.5 Section 8.1) queues events without loss; consumers resume from last offset on recovery
Database failover	RDS Multi-AZ failover triggered (Vol.7 Section 9.3)	Application reconnects within the Vol.7 Section 9.1 RTO target; no data loss within the RPO target
Poison message handling	A malformed event repeatedly fails consumer processing (Vol.3 Section 8.2)	Routed to dead-letter topic after the configured retry count; alert fires (Vol.7 Section 8.2)
Webhook endpoint failure (outbound)	A third-party integrator's webhook receiver is down (Vol.5 Section 7.1)	Exponential backoff retry for up to 24h, then dead-letter with visible delivery-log entry
Region/AZ loss simulation	Staging environment fails over to secondary AZ (Vol.7 Section 9.3)	Meets or approaches the RTO/RPO targets in a documented game-day exercise

11. Test Data Management

Environment	Data Approach
Local/dev, CI/ephemeral (Vol.7 Section 2.1)	Programmatically generated synthetic data, seeded fresh per test run; includes at least one multi-tenant scenario (2+ tenants with overlapping entity IDs) to make cross-tenant bugs visible by construction
Staging (Vol.7 Section 2.1)	Anonymized/synthetic data only - real tenant data is never copied to staging, consistent with Vol.6's compliance posture (Section 10)
Performance testing (Section 8)	Synthetically generated large-scale datasets modeling realistic transaction volume distributions (e.g. Pareto-distributed transaction counts per tenant) rather than uniform synthetic data, so load tests reflect realistic hot-tenant scenarios
Financial integrity suite (Section 9.2)	Deterministic, hand-crafted fixture data for known-answer testing (e.g. a fixture where the correct trial balance is computed independently and asserted against)

12. User Acceptance Testing & Acceptance Criteria

12.1 Acceptance Criteria Source

Per Vol. 1 Section 12's extraction note, functional acceptance criteria are derived directly from the Vol. 1 Section 6 Module & Feature Scope Catalog: every MVP-scoped row becomes a numbered acceptance criterion that must pass UAT before Release 1 ships. This avoids maintaining acceptance criteria as a separate, driftable document.

12.2 Sample Acceptance Criteria Derivation

Vol.1 Scope Row	Derived Acceptance Criterion
BC-02 Receivables MVP: invoices, quotes, payments received	AC-BC02-01: A user can create, send, and receive payment against an invoice, and the invoice status transitions correctly through the Vol.2 Section 8.1 state machine
BC-05 Banking MVP: manual statement import, reconciliation	AC-BC05-01: A user can import a bank statement file and reconcile at least one transaction against an existing invoice payment
BC-11 Tax Compliance MVP: single-country sales tax	AC-BC11-01: An invoice for a configured jurisdiction calculates tax correctly per the configured flat/tiered rate (Vol.5 Section 5.1)
BC-20 Security/Compliance MVP: RBAC, audit trail	AC-BC20-01: A user assigned the Sales Staff role (Vol.6 Section 3.3) cannot access Payables endpoints, and every financial mutation appears in the audit trail (Vol.6 Section 9)

12.3 UAT Sign-off Process

1. Each MVP acceptance criterion is verified against staging by a business stakeholder (or a designated proxy), not solely by the engineering team that built it.
2. A criterion is only marked accepted with a recorded pass note (screenshot or recorded session) attached to the criterion ID, ensuring an auditable UAT record exists.
3. Release 1 GA sign-off (Vol.7 Section 5.2 manual approval gate) requires 100% of MVP-tagged acceptance criteria accepted, with any exceptions explicitly waived in writing by the product owner.

13. Defect Management & Release Gates

13.1 Severity Classification

Severity	Definition	Release Impact
S1 - Blocker	Financial data corruption, cross-tenant data leakage, security vulnerability with active exploit path, complete inability to complete a core value stream (Vol.1 Section 7)	Blocks release; must be fixed and re-verified before any further release gate progresses
S2 - Major	Incorrect calculation not involving data corruption, a non-core feature completely broken, a workaround exists but is unreasonable for users	Blocks release unless explicitly waived by product owner with written rationale
S3 - Minor	Cosmetic issues, edge-case behavior with a reasonable workaround	Does not block release; tracked for a subsequent patch
S4 - Trivial	Typos, minor inconsistencies with no functional impact	Backlog, addressed opportunistically

13.2 Release Gate Checklist (feeds Vol. 7 Section 5.2 approval gate)

- All CI-blocking tests green (Section 2.2 layers, Section 7 security gates).
- Zero open S1 defects; all open S2 defects explicitly waived in writing.
- 100% of MVP-tagged UAT acceptance criteria (Section 12) accepted.
- Performance/load test results (Section 8) meet or exceed Vol.3 Section 12 targets on the current build.
- Most recent resilience/chaos test results (Section 10) show no regressions from the prior release.
- Security scan results (Section 7) reviewed with no unresolved high/critical findings.

14. Test Automation Tooling & CI Integration

Layer	Suggested Tooling	CI Integration Point (Vol.7 Section 5.1)
Unit (Section 3)	Jest (aligned with the TypeScript/NestJS stack, Vol.3 Section 2.1)	Lint + Unit stage
Integration (Section 4)	Jest + Testcontainers (real Postgres/Kafka/Redis in ephemeral containers)	Integration Tests stage
Contract (Section 5)	OpenAPI schema validation (e.g. Dredd/Spectral-class) + a schema registry client for event contracts	SAST/Contract stage, or a dedicated stage between Integration and E2E
E2E (Section 6)	Playwright (API-level and, for Vol.9 UI flows, browser-level) against staging	E2E stage, post-deploy-to-staging
Security (Section 7)	SAST/dependency scanner integrated into CI; DAST tool run on a schedule against staging; third-party pen test vendor engaged out-of-band	SAST stage (blocking) + scheduled DAST (non-blocking, tracked)
Performance (Section 8)	A load-testing tool (e.g. k6 or Locust-class) scripted against the Vol.3 Section 12 target scenarios	Pre-release stage, on-demand

15. Traceability & Next Phase

15.1 Traceability to Volumes 1-7

Vol. 8 Artifact	Traces To
Section 6 E2E Scenarios	Vol. 1 Section 7 Value Streams (VS-01...VS-06)
Section 7 Security Testing	Vol. 6 Section 12 Security Testing Requirements
Section 8 Performance Testing	Vol. 3 Section 12 Performance & Scalability Targets; Vol. 7 Section 7 Scaling Strategy
Section 9 Multi-Tenancy Suite	Vol. 4 Section 7 RLS; Vol. 6 Section 8 Defense-in-Depth
Section 10 Resilience Testing	Vol. 5 Section 8 Resilience Patterns; Vol. 7 Section 9 DR
Section 12 UAT Acceptance Criteria	Vol. 1 Section 6 Module & Feature Scope Catalog
Section 13 Release Gates	Vol. 7 Section 5.2 Deployment Strategy manual approval gate

15.2 Open Questions for Sign-off

4. Confirm whether a dedicated QA function/hire exists or is planned, or whether E2E test ownership (Section 2.2) rotates within engineering for Release 1.
5. Confirm the third-party penetration test vendor and timeline (Section 7) so it can be scheduled ahead of the targeted Release 1 GA date.
6. Confirm risk tolerance for waiving S2 defects (Section 13.1) at release - some teams require zero open S2s, others allow written waivers; this should be a deliberate choice, not a default.

15.3 Next Phase

On approval, Phase 9 (final) begins: UI/UX Design Specification & API Design Document (Volume 9) - completing the nine-volume blueprint with the design system, screen-level specifications for the value streams (Vol.1 Section 7), and full endpoint-by-endpoint API contracts building on Vol.2 Section 11's resource-ownership map and Vol.3 Section 5's API conventions.